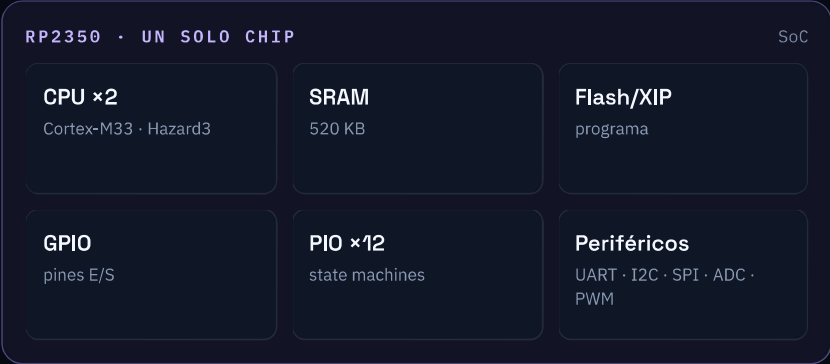


Arquitectura del hardware: del código al mundo físico

1 ¿Qué hay dentro del microcontrolador?



No es una mini PC. Un microcontrolador integra CPU, memoria y periféricos en un solo chip, sin sistema operativo tipo Linux, sin procesos y con memoria medida en kilobytes en lugar de gigabytes. La ingeniería embebida consiste en hacer mucho con muy poca memoria, y en saber exactamente dónde vive cada dato.

2 Mapa de memoria observado — direcciones reales



Lectura clave: Flash vive en el rango 0x10000000 mientras que globales, heap y stack conviven dentro de SRAM (0x20000000) pero en zonas separadas. Los periféricos ocupan rangos altos: 0x40000000 (APB), 0x50000000 (AHB) y 0xD0000000 (SIO, GPIO rápido).

3 ¿Qué es un registro? — el puente al mundo físico

Un registro es una dirección de memoria conectada directamente a hardware, no a una variable normal en RAM. Cada bit representa el estado de un pin, un temporizador o una interrupción. Escribir ese bit cambia algo físico de inmediato:

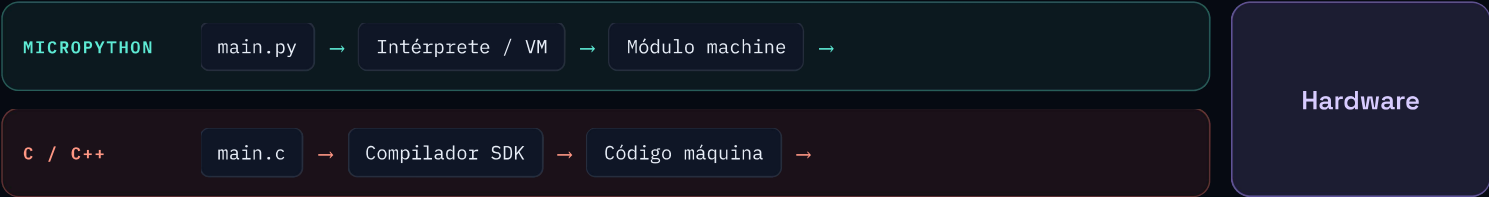
```
sio_hw->gpio_set = 1u << LED_PIN;
```

BIT = 0
≈0V
GPIO LOW

BIT = 1
≈3.3V
GPIO HIGH

CÓDIGO → CPU → BUS → REGISTRO → GPIO → 3.3V

4 Del programa al silicio: MicroPython vs C/C++



MicroPython traduce el código línea por línea en tiempo de ejecución y usa `machine` como puente a los registros: más lento, pero mucho más rápido de probar. C/C++ se compila a binario antes de correr; el SDK entrega funciones (`gpio_init`, `malloc`) que manipulan los registros por nosotros, pero permite ver las direcciones reales con `printf("%p", ...)`. **Abstracción = portabilidad:** el mismo concepto corre en ambas placas sin escribir direcciones hexadecimales a mano.

5 RP2040 vs RP2350

CARACTERÍSTICA	RP2040 · PICO W	RP2350 · PICO 2 W
CPU	2× Arm Cortex-M0+	2× Cortex-M33 o 2× Hazard3 RISC-V
SRAM	264 KB	520 KB
PIO (state machines)	8	12
Board target (SDK)	pico_w	pico2_w
Enfoque	Aprendizaje + IoT básico	Más memoria, seguridad y potencia
Rol en esta sesión	Chip simulado en Wokwi	Placa objetivo del curso

6 Evidencia de ejecución — simulador Wokwi

C/C++ CON PICO SDK

flash const: 10004A04
global var : 20001040
stack var : 20041FF4
heap ptr : 20001198

MICROPYTHON

libre inicial: 200848
después de reservar: 190832
después de liberar: 200848

Cada tipo de dato aparece en una región distinta. Se consumieron 10 016 bytes: los 10 000 del buffer más 16 bytes de overhead del objeto administrado por el recolector de basura, un costo que en C no existe.

7 Alcance y límites de la evidencia

No se utilizó hardware físico en esta entrega. Toda la evidencia se generó en el simulador Wokwi, que emula el chip RP2040 y no el RP2350 de la RASPBERRY PI PICO 2W